
LDCI2027: Light-Duty CI-ICE 2027

PROJECT ASSESSMENT 5

DEVELOPMENT OF AI-BASED OPERATION STRATEGY

REPORT 1

10TH APRIL 2026

FVV project 1507

Abstract

This report documents the current stage of the LDCI2027 work after a substantial methodological shift towards reinforcement learning (RL). Since the previous report phase, the repository and implementation workflow have been reworked around updated recurrent plant models, a new environment layer, and a broader experimental pipeline. The present document focuses on the integration of the new ICE and PG/drivetrain models, their efficient deployment in TensorFlow and ONNX, the development of a Gymnasium-compatible RL environment, and the implementation of training and evaluation workflows for several RL algorithms. Rather than presenting a final controller, the report emphasises the new narrative of the project: what became obsolete from the previous phase, what infrastructure has already been validated, and which intermediate results now guide the next development steps.

Index

Abstract	i
List of Figures	iii
Symbols	iv
1 Introduction	1
2 Project Evolution and Scope Update	2
3 Updated Plant Models	3
3.1 Inference Performance and Validation	5
4 Methodology	7
4.1 RL Problem Formulation	7
4.2 Experimental Approach	7
4.3 Implementation Details	8
4.3.1 Training	8
4.3.2 Evaluation	9
4.3.3 Environment	10
4.3.4 Manual Controller and Diagnostic Baseline	12
5 Intermediate Results	13
5.1 In General	13
5.2 Agent Results	13
5.2.1 PPO	14
5.2.2 SAC	15
5.2.3 TD3	17
6 Conclusion and Next Steps	19

List of Figures

1	Final neural network model setup	5
2	Runtime comparison between TensorFlow 2.15 and TensorFlow 2.18 for a full WLTC rollout of 3601 steps.	6
3	Additional deployment speedups obtained in the updated TensorFlow workflow.	6
4	Standardised six-phase target-velocity profile used as the default evaluation benchmark.	9
5	Screenshot of the manual controller interface used for diagnostic analysis.	12
6	Combined reward curves for PPO, SAC, and TD3 under the current Phase 1 speed-tracking setup.	14
7	PPO rollout diagnostics. Left: evaluation results for the learned PPO policy. Right: action trajectory and action-distribution view for the same PPO evaluation run.	15
8	SAC rollout diagnostics. Left: evaluation results for the learned SAC policy. Right: action trajectory and action-distribution view for the same SAC evaluation run.	16
9	TD3 rollout diagnostics. Left: evaluation results for the learned TD3 policy. Right: action trajectory and action-distribution view for the same TD3 evaluation run.	17

Symbols

Abbreviations

Abbreviation	Meaning
RL	Reinforcement Learning
ICE	Internal Combustion Engine
PG	Planetary Gear (referred to as Drivetrain)
LSTM	Long Short-Term Memory
PPO	Proximal Policy Optimization
SAC	Soft Actor-Critic
TD3	Twin Delayed Deep Deterministic Policy Gradient
SOC	State of Charge
POMDP	Partially Observable Markov Decision Process
MDP	Markov Decision Process
NOx	Nitrogen Oxides
WLTC	Worldwide Harmonized Light Vehicles Test Cycle
ONNX	Open Neural Network Exchange
DOC	Diesel Oxidation Catalyst
DPF	Diesel Particulate Filter
SCR	Selective Catalytic Reduction
CSV	Comma-Separated Values
MAE	Mean Absolute Error
RMSE	Root Mean Square Error

Mechanical Variables

Description	Python Variable	Units	Range
Engine speed	<code>ice_sp</code>	rpm	0 – 4500
Fuel mass	<code>mf</code>	mg	3 – 70
Ambient temperature	<code>T_amb_K</code>	K	250 – 310
Ambient pressure	<code>p_amb_bar</code>	bar	0.8 – 1.1
ICE torque	<code>torque</code>	Nm	–50 – 300
Nitrogen oxides	<code>Nox</code>	mg/s	-
EM2 torque	<code>EM2</code>	Nm	–421 – 421
Brake percentage	<code>brk</code>	%	0 – 100
State of Charge	<code>soc</code>	%	0 – 100
Car velocity	<code>vel_out</code>	km/h	-

1 Introduction

This report documents the current state of the LDCI2027 work after a methodological and implementation reset. The previous report structure mainly reflected an earlier phase centred on a predictive controller built around surrogate models of the internal combustion engine (ICE) and the hybrid powertrain/PG (in the following referred as drivetrain). Since then, both the repository and the development strategy have changed significantly.

The problem remains the same at a high level: to control the drivetrain in a way that achieves accurate speed tracking while considering additional objectives such as emissions and battery state of charge. However, the current stage of the project is no longer best described as the continuation of the earlier controller architecture. It is now more accurately framed as the development of a reinforcement learning (RL) workflow built on updated plant models, a new environment layer, and a unified experimentation pipeline.

The present document therefore focuses on three aspects: updated plant models, the design of the RL environment and training workflow, and the intermediate results obtained so far. Rather than claiming a final controller, it presents the infrastructure, methodological decisions, and validated progress that define the current phase of the project.

2 Project Evolution and Scope Update

During the execution of this task, new versions of the ICE and drivetrain models were received and several project assumptions had to be revised. In the earlier phase, much of the effort was invested in a predictive controller and in the technical adaptations required by successive model updates. That phase remains useful as background, but it is no longer the centre of the present report.

The current stage differs in three essential ways:

- The plant-model stack has changed, including new LSTM models, updated validity ranges, and a specific initialisation procedure.
- The computational efficiency has improved, enabling broader RL experimentation.
- The research scope has expanded from a narrow TD3-centred perspective to a comparative pipeline including multiple RL algorithms.

This change in scope affects the organisation of this report as well. Chapters that were previously devoted to the detailed internals of the older controller are now mainly obsolete and can be considered secondary, while chapters on environment design, reward engineering, algorithm comparison, and intermediate RL results become central. However, for further insights into what preliminary project findings were, please refer to the old report from PA4.

3 Updated Plant Models

The control framework is built on two recurrent surrogate models representing the internal combustion engine (ICE) and the drivetrain. These models replace more expensive physical simulations while preserving the temporal structure needed for control development.

In the current project phase, an updated version of the long short-term memory (LSTM) models provided by Markus Frey on February 3, 2026, is used. According to the implementation documentation, no fundamental changes were made to the underlying numerical plant models. The main changes relate to validity ranges, model initialisation, and deployment. Compared with the models described in the earlier PA4 stage, the most relevant updates are the following:

- The ICE model now has **two distinct valid operating regimes**: engine-off operation with $n_{ICE} = 0$ rpm and $m_{fuel} = 0$, and engine-on operation in the range $900 \text{ rpm} \leq n_{ICE} \leq 4000 \text{ rpm}$ with $3 \text{ mg} \leq m_{fuel} \leq 70 \text{ mg}$.
- The intermediate region between engine-off and engine-on conditions is **explicitly invalid**. In particular, operation between 0 and 900 rpm or between 0 and 3 mg fuel mass is not supported and may produce unphysical behaviour.
- The drivetrain model preserves the previous structure, but its validated operating range is stated explicitly: state of charge (SoC) $\in [0, 1]$, $n_{ICE} \in [0, 4500]$ rpm, $\tau_{EM2} \in [-421, 421]$ Nm, $\tau_{ICE} \in [-50, 300]$ Nm, and mechanical braking in the range 0–100%.
- All recurrent models now use a **two-part LSTM deployment scheme**, with a dedicated initialisation model (`model_init.keras`) and an inference model (`model_inf.keras`).
- The application workflow was updated to **TensorFlow 2.18**, and the models are loaded once and then reused over the complete driving cycle, which reduces avoidable inference overhead.

The operating ranges and the network interfaces are summarised in Table 3, Table 4, Table 5 and Table 6. The final three-network architecture is displayed in Figure 1.

Parameter	Range
Speed (n_{ICE})	Engine off: 0 rpm; engine on: 900–4000 rpm
Fuel mass (m_{fuel})	Engine off: 0 mg; engine on: 3–70 mg
Ambient temperature (T_{amb})	250–310 K
Ambient pressure (p_{amb})	0.8–1.1 bar

Table 3: Valid ranges of the ICE model

Parameter	Range
State of charge (SoC)	0–1
Engine speed (n_{ICE})	0–4500 rpm
EM2 torque (τ_{EM2})	–421 to 421 Nm
ICE torque (τ_{ICE})	–50 to 300 Nm
Mechanical brake (Brakemech)	0–100%

Table 4: Valid ranges of the drivetrain model

A particularly important point is the use of dedicated initialisation networks. The updated implementation does not start recurrent inference from arbitrary hidden states. Instead, the initial timestep is handled by a separate initialisation model that maps physically meaningful initial conditions to the internal LSTM state required by the inference model. This directly affects the environment reset logic, because a reset must initialise not only the observable state variables but also the recurrent memory in a way that is consistent with the plant-model hidden state.

The documentation also mentions an additional **Drivetrain Plus** network, whose interface is summarised together with the standard drivetrain model in Table 6. It uses the same inputs as the standard drivetrain model, namely n_{ICE} , τ_{ICE} , τ_{EM2} , and **Brakemech**, but predicts extra internal drivetrain quantities such as actual EM1 torque, actual EM2 torque, and sun and ring speeds. It is not required

Group	Parameter	Description
Input	n_{ICE} [rpm]	Control input to all neural networks
Input	m_{fuel} [mg/cyl/cycle]	Fuel-mass control input
Input	T_{amb} [K]	Boundary input
Input	p_{amb} [bar]	Boundary input
Output	τ_{ICE} [Nm]	Transfer output to the drivetrain model
Output	$m_{fuel_{tot}}$ [g/s]	Total burned fuel mass
Output	NOx_{eo} [g/s]	Raw emission
Output	CO_{eo} [g/s]	Raw emission
Output	THC_{eo} [g/s]	Raw emission
Output	$T_{gas_{eo}}$ [K]	Engine-out gas temperature
Output	NOx_{tp} [g/s]	Tailpipe emission
Output	CO_{tp} [g/s]	Tailpipe emission
Output	$CO2_{tp}$ [g/s]	Tailpipe emission
Output	THC_{tp} [g/s]	Tailpipe emission
Output	$T_{wall_{SCR1}}$ [K]	SCR1 wall temperature
Output	$T_{wall_{DOC}}$ [K]	DOC wall temperature
Output	$T_{sub_{DPF}}$ [K]	DPF wall temperature
Output	$T_{wall_{SCR2}}$ [K]	SCR2 wall temperature
Output	$T_{wall_{SCR3}}$ [K]	SCR3 wall temperature
Output	$T_{gas_{tp}}$ [K]	Tailpipe gas temperature

Table 5: Inputs and outputs of the ICE neural network

Network	Group	Parameter	Description
Drivetrain	Input	n_{ICE} [rpm]	Control input to all neural networks
Drivetrain	Input	τ_{ICE} [Nm]	Transfer input from ICE
Drivetrain	Input	τ_{EM2} [Nm]	Control input
Drivetrain	Input	Brakemech [%]	Control input
Drivetrain	Output	v_{Car} [km/h]	Vehicle speed
Drivetrain	Output	SoC [-]	Battery state of charge
Drivetrain Plus	Input	n_{ICE} [rpm]	Control input to all neural networks
Drivetrain Plus	Input	τ_{ICE} [Nm]	Transfer input from ICE
Drivetrain Plus	Input	τ_{EM2} [Nm]	Control input
Drivetrain Plus	Input	Brakemech [%]	Control input
Drivetrain Plus	Output	$\tau_{EM1,out}$ [Nm]	Actual EM1 torque
Drivetrain Plus	Output	$\tau_{EM2,out}$ [Nm]	Actual EM2 torque
Drivetrain Plus	Output	n_{Sun} [rpm]	Actual sun speed
Drivetrain Plus	Output	n_{Ring} [rpm]	Actual ring speed

Table 6: Inputs and outputs of the drivetrain and Drivetrain Plus neural networks

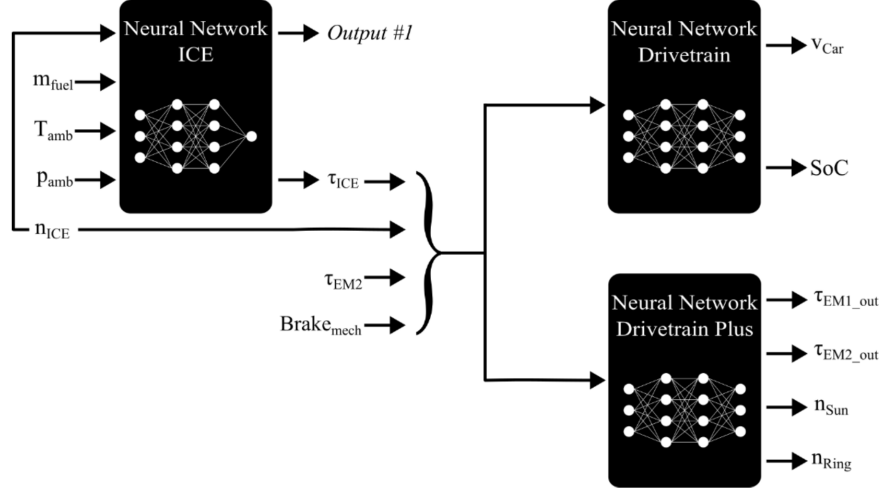


Figure 1: Final neural network model setup

for the present RL control loop, but it is potentially valuable for future work because it could provide richer state diagnostics and more interpretable post-analysis of drivetrain operation.

More concretely, the final model stack consists of three neural networks: an **ICE network**, a **Drivetrain network**, and the optional **Drivetrain Plus network**, as sketched in Figure 1. The ICE network receives engine speed, fuel mass, ambient temperature, and ambient pressure as inputs and predicts engine torque as the transfer output, together with emissions- and aftertreatment-related variables, as listed in Table 5. The documentation emphasises that the ICE model is still based on the EA189 engine because test-bench measurement data is not yet available. One important extension in the new dataset is the addition of stationary cooldown cycles, which allows cooldown behaviour during engine-off phases to be represented. This is the reason why the new ICE validity range must be interpreted as two separate operating regimes instead of one continuous interval.

The drivetrain network receives engine speed, ICE torque, EM2 torque, and mechanical braking as inputs, and predicts vehicle speed and battery state of charge, as summarised in Table 6. For RL purposes, these are the key plant outputs required to evaluate tracking performance, energy usage, and feasibility of the control action. Ambient conditions are assumed to remain constant over one complete driving cycle of 1800 seconds, with valid ranges of 250–310 K for ambient temperature and 0.8–1.1 bar for ambient pressure.

3.1 Inference Performance and Validation

One of the major limitations of the previous workflow was execution speed. Earlier integration choices created an avoidable wall-time bottleneck during simulation, which strongly limited the practicality of longer control experiments.

In the current setup, this bottleneck was reduced by using TensorFlow graph execution, which avoids inefficient per-step handling and improves simulation throughput. This change is major, since RL training depends directly on the rate at which inference can be done since data to be trained on is created on the fly. A first benchmark already shows the magnitude of the gain obtained by moving from TensorFlow 2.15 to TensorFlow 2.18 for the Worldwide Harmonized Light Vehicles Test Cycle (WLTC). As shown in Figure 2, the total WLTC rollout time decreases from 104.50 s to 4.79 s, and the average cost per step decreases from 29.02 ms to 1.33 ms (conducted locally on a MacBook).

On top of the TensorFlow implementation, Marcial Francisco Mosqueda Otero developed Open Neural Network Exchange (ONNX) versions of the plant models to further improve efficiency. Profiling results for scalar handling and model execution are shown in Figure 3 (conducted by Marcial, no information about the machine). In addition to speed, output consistency between the TensorFlow 2.18 and

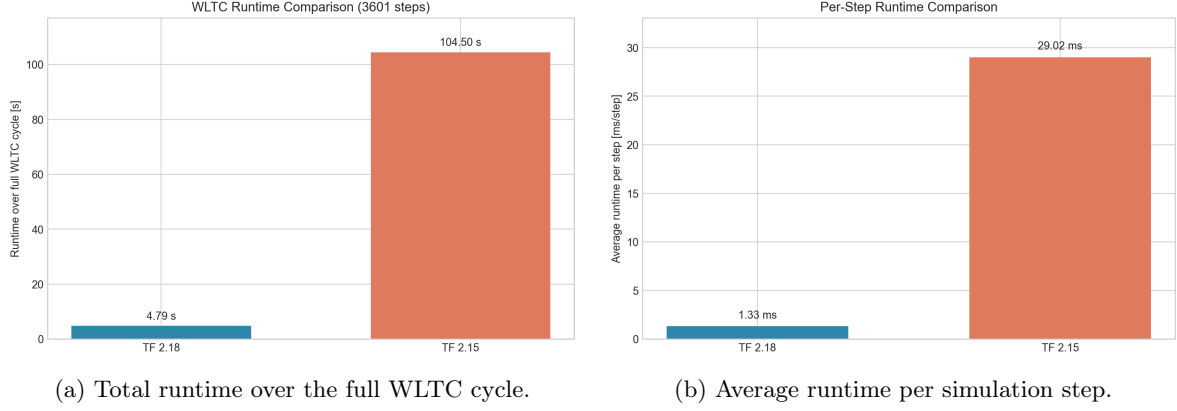


Figure 2: Runtime comparison between TensorFlow 2.15 and TensorFlow 2.18 for a full WLTC rollout of 3601 steps.

ONNX implementations was checked over 3601 samples from a full WLTC rollout. The results in Table 7 show that the discrepancies are negligible for the main variables of interest, with maximum absolute differences on the order of 10^{-4} or below.

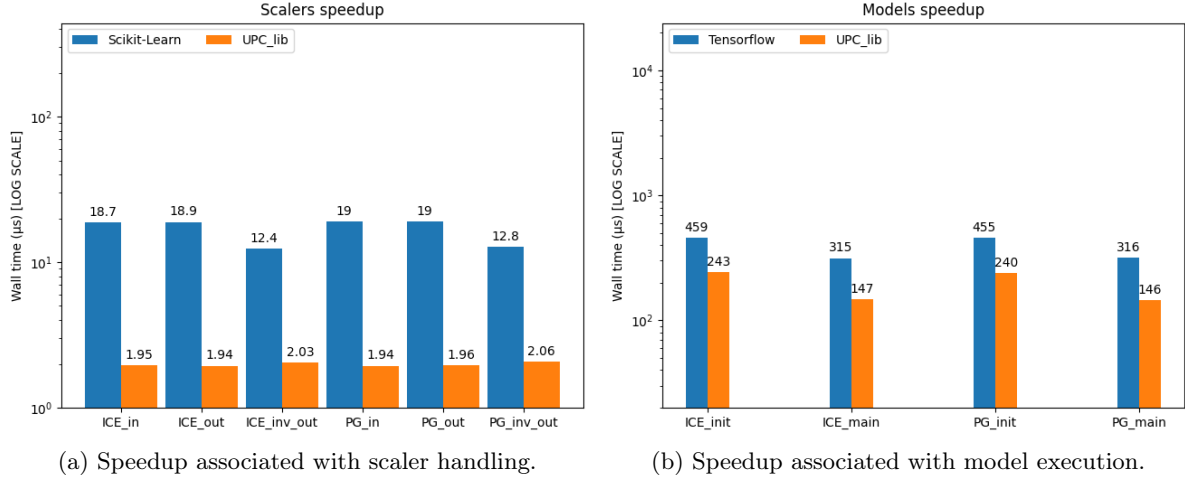


Figure 3: Additional deployment speedups obtained in the updated TensorFlow workflow.

Column	Max difference	MAE	RMSE	Samples
car_speed_pred	0.000124522706912	3.43454772865e-05	4.30399170608e-05	3601
ice_torque_pred	0.00016313820669	3.31201547304e-05	4.26499684198e-05	3601
soc_pred	1.72454509073e-06	9.15798414726e-07	1.00679028928e-06	3601

Table 7: Consistency checks between the TensorFlow 2.18 and ONNX plant implementations over 3601 samples.

4 Methodology

4.1 RL Problem Formulation

The task is formulated as a reinforcement learning problem in which an agent interacts with the plant-model stack through a Gymnasium-compatible environment.

The developed environment was reviewed together with Markus Frey in order to reduce the risk of major conceptual errors arising from an incomplete understanding of the underlying vehicle-engineering constraints. It is ensured that the environment follows the updated logic for initialisation and validity restrictions introduced in section 3.

The formulation is naturally partially observable because the internal recurrent states of the plant models, which were intended to capture thermal engine dynamics, are not measurable physical variables. For this reason, the control problem is more accurately interpreted as a partially observable sequential decision problem than as a fully observed one. However, at the current stage of development and for the sake of reducing complexity, these thermal effects are not yet considered explicitly. The resulting problem is therefore treated as a Markov decision process, even though this constitutes a simplifying approximation of the underlying POMDP.

The current work is no longer limited to TD3. Instead, the project now follows a comparative RL pipeline in which TD3, PPO, and SAC are trained and evaluated under a common environment interface. The implementations are based on **Stable-Baselines3**, which provides a consistent framework for working with RL agents.

This broader comparison is important because it would be premature to assume that a single algorithm family necessarily offers the best trade-off between training stability, sample efficiency, and final control quality.

4.2 Experimental Approach

In the early stage of development, much of the work focused on reward tuning and on observing how the agents reacted to different combinations of reward terms. Although this exploratory phase was useful for building intuition, it did not yet constitute a sufficiently structured methodology. Given the stochastic nature of RL training and the limited computational budget, ad hoc reward adjustments alone would have made it difficult to draw reliable conclusions.

After re-aligning the methodology with Cecilio Angulo Bahun, the experimental strategy was reformulated as a phase-wise procedure, conceptually close to curriculum learning. The goal is to separate basic controllability from later multi-objective optimisation and thus make the validation process more interpretable.

- **Phase 1:** The agents are trained with a single active objective, namely speed tracking. In this stage, PPO, SAC, and TD3 are each expected to learn how to follow the desired speed profile as accurately as possible, while effects on battery state of charge (SOC) and emissions are deliberately ignored in the reward. At the same time, the resulting trajectories are used to record the corresponding SOC variation and emission levels, thereby establishing empirical reference bounds for the later optimisation stages.
- **Phase 2:** Once speed tracking has been validated, the reward is extended to include additional objectives related to emissions and SOC preservation. Emissions cannot be activated in isolation, because such a formulation would likely encourage the agent to rely almost exclusively on electric propulsion. For this reason, the SOC-difference term must be activated as well, so that the learned policy remains charge-sustaining or at least avoids unrealistic battery depletion. The central methodological question in this phase is to determine how far the non-speed reward terms can be strengthened while maintaining acceptable tracking quality. Regarding emissions, only NOx is considered for now while other emissions (e.g. CO, CO2, THC) might be included later on.

An acceptance interval for speed tracking must therefore be defined explicitly. At present, a reasonable example is a tolerance interval of ± 5 km/h around the reference trajectory, although the final threshold

will be stated and justified in the final documentation. Within that admissible interval, the secondary objectives can then be tuned in a systematic way.

The transition from Phase 1 to Phase 2 can be handled in two ways. One possibility is to treat Phase 1 as pre-training, continue from the learned weights, and refine the policy under the richer reward. The second possibility is to use Phase 1 only as a diagnostic stage for identifying feasible SOC and emissions ranges, and then retrain the agents from scratch in Phase 2. Both options are methodologically defensible and should be compared in terms of stability and final performance.

Throughout both Phase 1 and Phase 2, hyperparameter optimisation is planned for all three agents. For this purpose, the Optuna framework will be used to analyse hyperparameter sensitivity and to identify suitable configurations for each algorithm and experimental setting. This process may also include dynamic training adjustments, for example a gradual reduction of entropy-related parameters as training progresses.

The expected outcome of this experimental programme is a trained RL agent that can dynamically follow a target speed profile while simultaneously reducing emissions and maintaining SOC within acceptable limits. In order to reduce the influence of randomness and support fair comparison between algorithms, each relevant experiment should be repeated over multiple seeds, with approximately five to ten runs per configuration, and results should be reported as averaged performance with an appropriate measure of spread.

4.3 Implementation Details

This subsection outlines highlights of the practical implementation of training, evaluating, and analysing the RL agents. Furthermore, it provides a comprehensive description of the developed RL environment, which represents the elementary part of the development.

4.3.1 Training

The training pipeline handles parameter and hyperparameter loading from configuration files, stores the effective training configuration for reproducibility, records intermediate results, tracks wall-clock performance, and saves the final trained model together with the associated metadata. For on-policy training, the pipeline also stores the normalisation statistics required for consistent evaluation.

Six-step target velocity trajectory: At present, the main training setup is based on six-step target-velocity profiles over 1800 seconds, corresponding to 3600 simulation steps with 600 steps per phase. This choice provides a controlled intermediate level of difficulty and has therefore become the current default for development. However, the pipeline also supports episodes with a single target velocity and full speed cycle, including synthetic speed cycles related to the distribution used for plant-model training and real-world cycles as the WLTC. This makes it possible to progress from simple to more complex tasks during development and later to assess generalisation under more realistic operating conditions.

On-/Off-policy differences: From an implementation perspective, off-policy and on-policy algorithms are handled differently. Off-policy methods use the static scaling defined directly in the environment (see subsection 4.3.3), which avoids introducing a moving normalisation target into the replay buffer. By contrast, the on-policy PPO pipeline can additionally exploit dynamic observation and reward normalisation through `VecNormalize` in `Stable-Baselines3` since the algorithm is discarding the collected experience after using it. Training checkpoints are stored every 100000 steps, and for the normalised on-policy runs the associated statistics are saved in `.pk1` files so that evaluation remains consistent with training.

Parallelised training: Training is parallelised through `SubprocVecEnv`, also from `Stable-Baselines3`, so that multiple environments can be executed concurrently on separate CPU cores. This improves data collection throughput, which is especially relevant for RL because trajectories must be generated online during training.

In addition to optimisation itself, the training pipeline also includes reward plotting, entropy-related diagnostics, and runtime monitoring, so that convergence behaviour and exploration properties can be inspected.

4.3.2 Evaluation

The evaluation pipeline applies a selected trained policy, loaded from the designated weight path, to a corresponding evaluation environment wrapped in a `DummyVecEnv`. In this phase, inference is performed deterministically so that the agent’s behaviour can be analysed without the additional variability introduced by exploration noise. Different reference trajectories can be used for evaluation, including synthetic cycles, real-world cycles, and constant-speed scenarios. At present, however, the default benchmark remains a standardised six-phase cycle, illustrated in Figure 4.

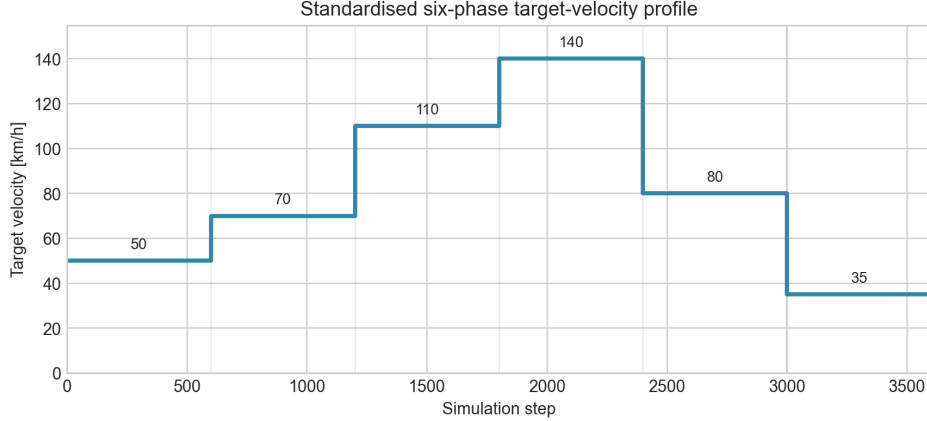


Figure 4: Standardised six-phase target-velocity profile used as the default evaluation benchmark.

Quantitative evaluation: For each evaluation rollout, a common set of summary metrics is computed. These include the total accumulated reward, total fuel consumption, total NOx emissions, mean absolute speed-tracking error, root-mean-square speed-tracking error, initial SOC, final SOC, and the resulting SOC difference over the trajectory. Together, these quantities provide a compact description of control quality, energy usage, and the side effects of the learned policy.

Visual qualitative evaluation: The evaluation stage also produces several diagnostic visualisations. These include state–action occupancy plots, one-dimensional and two-dimensional state-visitation maps, and time-series plots of the actions chosen by the agent over the full evaluation horizon together with their empirical distributions. Such outputs are important because they make it possible to distinguish between merely acceptable scalar metrics and genuinely plausible control behaviour. In particular, they help identify whether an apparently successful policy relies on pathological action patterns, unrealistic switching behaviour, or narrow regions of the admissible state space.

Emission assesment: In addition, the evaluation pipeline exports a file containing emissions per kilometre. This artefact originates from earlier work on WLTC-based testing, but it remains useful for the present project because it provides a direct bridge to application-oriented assessment. Once real driving cycles such as the WLTC are reintroduced more systematically, this output will again become important for judging whether the learned policies remain compatible with relevant emissions limits such as the EURO 6 or EURO 7 targets.

The current experiment matrix is summarised in Table 8. At this stage, all three algorithms are trained on the same family of six-step target-velocity trajectories and evaluated on the same standardised benchmark cycle. This keeps the comparison focused on algorithmic differences and on the effect of the normalisation strategy rather than on differences in scenario design.

Algorithm	Training Speed Cycle	Evaluation Speed Cycle	Static Scaling	Dynamic Normalisation
PPO (on-policy)	6 steps randomly sampled between 0 and 150 km/h	Standardised evaluation trajectory	Yes	Yes
SAC (off-policy)	6 steps randomly sampled between 0 and 150 km/h	Standardised evaluation trajectory	Yes	No
TD3 (off-policy)	6 steps randomly sampled between 0 and 150 km/h	Standardised evaluation trajectory	Yes	No

Table 8: Current experiment matrix used for the comparative RL study.

4.3.3 Environment

The environment is the central component of the RL implementation because it defines the interface between the plant-model LSTMs and the RL agents acting on them. Accordingly, the implemented `env.py` contains the restrictions, control logic, and validity constraints introduced by the updated plant LSTM models (section 3).

Actions: The current action space consists of four variables, namely `[ICE_Command, EM2_Torque, Fuel_Mass, Brake]`, each normalised to the interval $[-1, 1]$ at the agent interface. Internally, these actions are mapped back to physically meaningful ranges. In the current implementation, `ICE_Command` is handled separately inside the environment step logic, while the remaining actions are bounded according to the valid operating range of the plant models. Concretely, the corresponding lower and upper bounds are $[-1.0, -421.0, 3.0, 0.0]$ and $[1.0, 421.0, 70.0, 100.0]$. These limits are derived from the validity ranges of the latest plant LSTM models (see section 3). In an earlier version, a fifth action was introduced solely to encode engine on/off behaviour; however, this approach was later replaced by the more integrated formulation described below in the paragraph on engine on/off encoding.

Observations: The observation space consists of nine variables, encoding vehicle speed, speed error, SOC, ICE torque, NOx, ICE speed, fuel, SOC error, and a normalised timer variable. In implementation terms, this observation space is represented by a `spaces.Box` with lower bounds $[-5.0, -155.0, 0.0, -50.0, 0.0, 900.0, 3.0, -1.0, 0.0]$ and upper bounds $[150.0, 155.0, 1.0, 300.0, 0.4, 4000.0, 70.0, 1.0, 1.0]$. These bounds are either taken directly from the validity ranges provided for the plant models or derived manually from physical and modelling considerations.

Rewards: Rewards are scaled using heuristically determined maximum values for speed- and emissions-related terms, together with logically defined maximum values for fuel and brake terms. In its most general form, the reward can be written as

$$R_t = \underbrace{W_{\text{speed}} \cdot \exp\left(-\frac{1}{2} \left(\frac{\Delta v}{\sigma}\right)^2\right)}_{\text{Speed Tracking}} - \underbrace{(W_{\text{emis}} \cdot \tilde{E}_{\text{NOx}})}_{\text{Emissions}} - \underbrace{(W_{\text{soc}} \cdot |\Delta \text{SOC}|)}_{\text{SOC Management}} - P_{\text{mech}}.$$

This decomposition is helpful because it makes explicit how the different control objectives enter the learning signal. At present, however, not all of these terms are active simultaneously. In the current Phase 1 setting, the only active reward component is the speed-tracking term, whereas the emissions and SOC-related terms are intended for Phase 2.

The speed-tracking term is formulated as a squared-exponential (Gaussian) reward. Here, Δv denotes the absolute speed error, corresponding to `speed_error`, and σ defines the width of the Gaussian curve. In the current implementation, this scale factor is set to 10. The resulting reward reaches its maximum when the vehicle speed matches the reference speed and decreases smoothly as the tracking error grows.

The emissions term is based on the normalised NOx quantity $\tilde{E}_{\text{NOx}}$, which corresponds to the capped emission penalty used in the implementation. At the current stage, only NOx is considered, while additional pollutants may be incorporated later. The SOC-management term is based on the deviation from the initial battery state, denoted by ΔSOC and corresponding to `soc_error`. This term has not yet been finalised, since it remains open whether the final formulation should penalise SOC deviation linearly or quadratically.

In addition, a background regularisation term is used to discourage mechanically implausible behaviour:

$$P_{\text{mech}} = W_{\text{brake}} \cdot \tilde{B} + W_{\text{flicker}} \cdot \delta_{\text{start}},$$

where $\tilde{B}$ denotes normalised mechanical brake usage and δ_{start} is a binary indicator that becomes active when the engine switches from off to on. These shaping terms were introduced to guide training more effectively, since the agents otherwise struggled with the relatively high-dimensional continuous action space. The corresponding shaping terms are defined in `config_rewards.py`, which is standardised for all three algorithms during training.

Static scaling: Neural networks generally require inputs and rewards to be scaled to small, relatively uniform ranges in order to prevent gradient explosion and support stable optimisation. Dynamic normalisation tools such as `VecNormalize` from `Stable-Baselines3` can serve this purpose, but they are only suitable for algorithms that do not retain past experiences, that is, on-policy methods. For off-policy algorithms, dynamic normalisation induces a continuous drift in the semantic meaning of states and rewards as more data are collected. For example, an engine-speed value normalised during an early phase of training would no longer correspond to the same effective scale later on. When such experiences are sampled again from the replay buffer, this drift can corrupt the stored data and destabilise critic training. For this reason, `VecNormalize` is only used for PPO.

To maintain comparable input ranges across all algorithms, a static scaling procedure was instead implemented directly in the environment. Rewards are scaled by constant factors, as described above. This guarantees stable, low-variance inputs without distorting the underlying physical meaning of the simulation.

During the preparation of this report, it was noticed that observations are not yet scaled correctly in the current implementation. The definition of the observation space through `spaces.Box` serves documentation and Gymnasium metadata purposes, but it does not automatically scale the observations. It would therefore be inaccurate to claim that proper observation scaling is already in place. Since this issue has now been identified, it will be addressed in the next development iteration. Observations will then be min-max scaled to a fixed $[-1, 1]$ range using predefined physical boundaries.

Shielding: Charging and discharging are manually blocked when $\text{SOC} \geq 0.98$ and `em2_torque` < 0 , or when $\text{SOC} \leq 0.02$ and `em2_torque` > 0 . This serves two purposes: first, it prevents invalid inference because the LSTM plant models are not valid outside this SOC range, in particular at SOC values of 0 or 1; second, it avoids unhealthy battery operation, since fully charged or deeply discharged states are detrimental to battery longevity.

Valid initial conditions via CSV sampling: To preserve a degree of generalisation despite the introduction of six-step velocity trajectories, the initial conditions, in particular vehicle speed and SOC, are varied by sampling from the synthetic CSV files that Markus Frey used for training the LSTM plant models. This makes the training distribution more diverse and therefore supports the learning of a more robust policy.

Engine switch cooldown: To represent the physical constraint that the engine cannot be switched on and then immediately switched off again in the next step, Markus Frey proposed a heuristic cooldown time of **3 seconds**. This restriction was implemented manually in the step function, ensuring that the agent cannot toggle the engine state within that interval. However, because this behaviour is enforced inside the environment rather than controlled directly by the agent, the originally framed MDP acquires a partially observable character. To compensate for this, the corresponding timing information is included in the observation vector. Concretely, the number of steps since the last engine-state toggle is counted and divided by 6, since 6 steps correspond to 3 seconds, thereby yielding a scaled quantity between 0 and 1.

Encoding of engine on/off: Initially, an additional action variable was introduced to encode engine on/off decisions through a discretised continuous signal in the interval $[-1, 1]$, with 0 acting as the switching threshold. In the current design, however, the original engine-speed action has been replaced

by this encoding directly: values below 0 correspond to engine-off behaviour, in which fuel and ICE speed are manually set to zero, whereas values above 0 are linearly interpolated onto the valid ICE-speed range from 900 to 4000 rpm. This formulation is more compact, but it also introduces a sharp transition around 0. As a result, small changes around the threshold produce abrupt changes in the environment dynamics, which leads to a non-smooth optimisation landscape for gradient-based learning. A plausible hypothesis is that deterministic algorithms such as TD3 are particularly sensitive to this characteristic, as discussed further in subsection 5.2.3.

4.3.4 Manual Controller and Diagnostic Baseline

In addition to the RL development, a manual controller was implemented as a diagnostic tool. Although it is not the target methodology, it provides three practical benefits: it offers a baseline for checking whether the simulator can produce coherent trajectories, it helps build intuition for plant behaviour under constraints such as engine on/off transitions and SOC limits, and it may later serve as a source of demonstrations for imitation learning or policy warm starts.

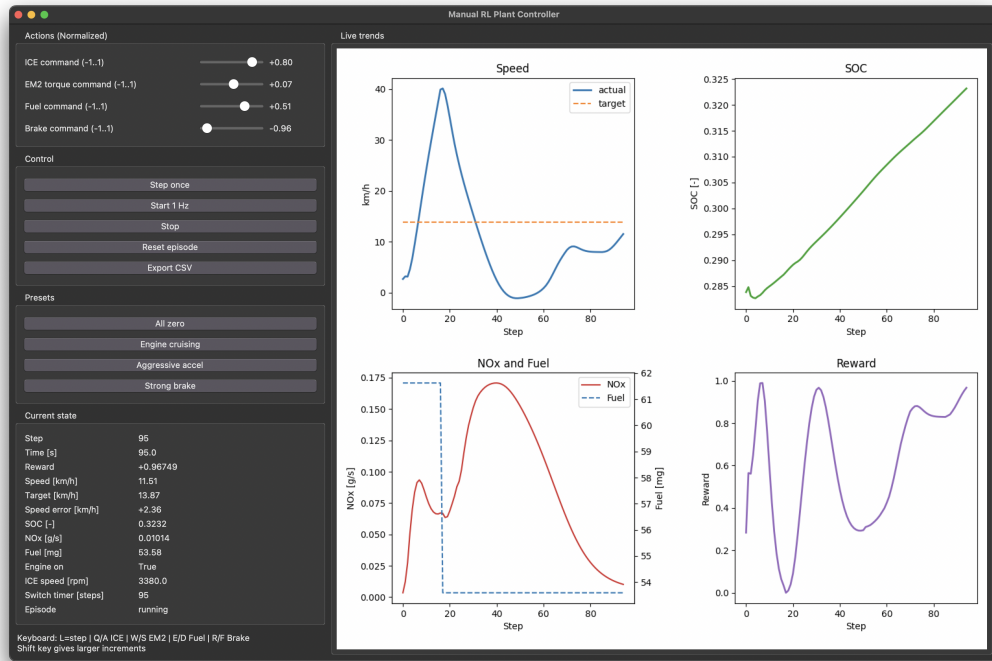


Figure 5: Screenshot of the manual controller interface used for diagnostic analysis.

5 Intermediate Results

This section describes intermediate findings at the current stage. Those should not be considered final and are only supposed to confirm that the overall experimental pipeline is technically viable.

5.1 In General

On the plant-model side, the new architectures have led to a substantial improvement in experimentation speed. Whereas the earlier work reported that only around **300 interactions** could be sampled within one hour, the current implementation permits approximately **4 million** interaction steps for PPO within 40 minutes. This figure covers the complete rollout process, including inference through the plant-model LSTMs as well as the forward and optimisation steps of the PPO agent. Such a speed-up is an obligatory precondition for RL experimentation and therefore already constitutes an important technical result.

On the environment side, the implementation is now functionally aligned to the latest requirements and supports controlled rollouts through the intended Gymnasium-style interface. At the agent level, the main result so far is that the full experimentation loop is operational: agents can be trained on different speed-trajectories, evaluated systematically, and exposed to reward terms that are activated progressively in accordance with the staged methodology.

At present, the main limitation is not conceptual but computational. Access to the cluster is still required before extensive multi-seed experiments and systematic hyperparameter optimisation with Optuna can be carried out. Nevertheless, local runs without statistical averaging have already produced encouraging results for Phase 1 (see section 4). In particular, PPO and SAC were both able to learn speed-control policies that follow the evaluation trajectory closely: in the representative runs reported later, PPO achieved a mean absolute speed-tracking error of 2.05 km/h and an RMSE of 3.52 km/h, while SAC achieved a mean absolute speed-tracking error of 0.81 km/h and an RMSE of 4.15 km/h. These initial results were obtained using the default hyperparameters provided by **Stable-Baselines3**.

5.2 Agent Results

This subsection summarises the current behaviour of the three RL algorithms under the Phase 1 speed-tracking objective. These results should be interpreted as preliminary evidence rather than as final experimental conclusions, since they do not yet include statistical averaging across seeds or systematic hyperparameter optimisation. Even so, they are already useful for identifying broad differences in learning dynamics, control quality, and behavioural plausibility.

The reward trajectories of PPO, SAC, and TD3 are compared in Figure 6. This combined view is particularly helpful at the present stage because it highlights qualitative differences in convergence speed, stability, and asymptotic performance before turning to the more detailed rollout-level diagnostics for each algorithm.

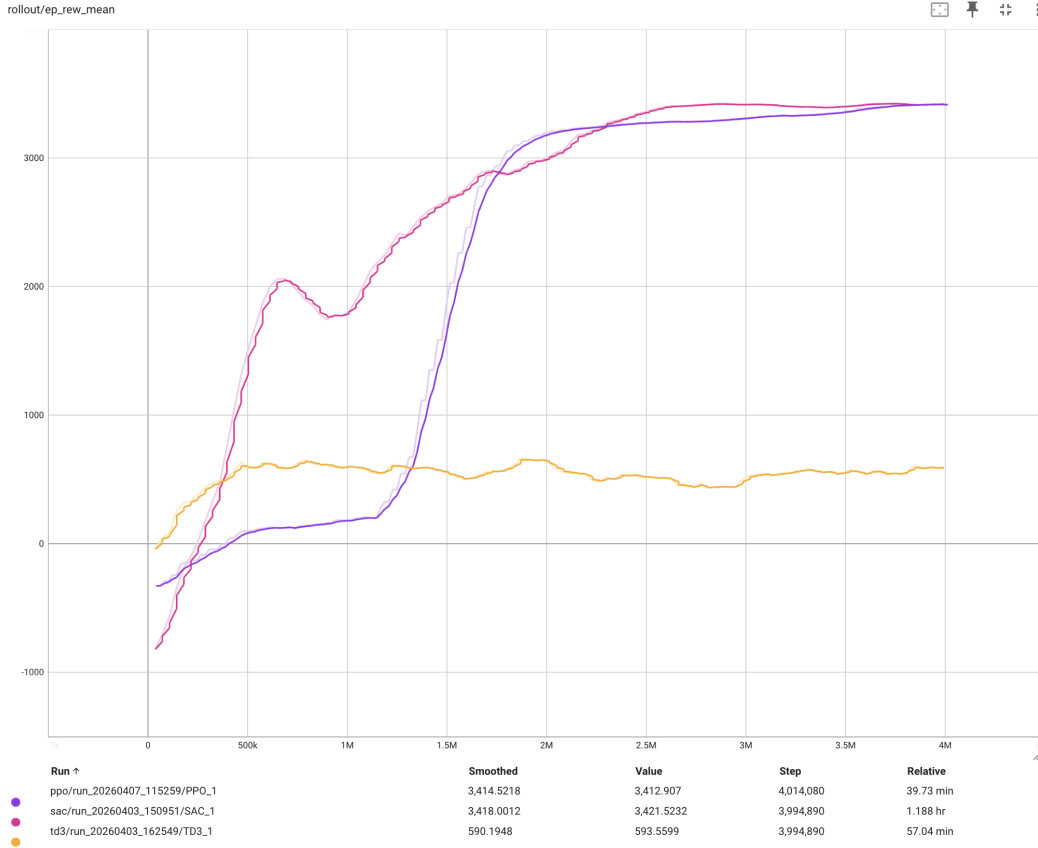


Figure 6: Combined reward curves for PPO, SAC, and TD3 under the current Phase 1 speed-tracking setup.

5.2.1 PPO

Among the three algorithms, PPO produced one of the strongest and most stable results observed so far. Its best run reached a mean episodic training reward of 3412.91, averaged over 100 episodes. For reference, a value of 3600 would correspond to perfect control over the full episode under the current Gaussian speed-tracking reward, since an exactly matched speed profile yields a reward contribution of +1 at each step. As already visible in Figure 6, PPO shows a strong convergence tendency towards high-quality speed tracking.

This result indicates that PPO is able to learn the intended Phase 1 objective of near-perfect speed tracking. The run discussed here was obtained with a small adjustment to the brake and flicker penalty terms, each reduced from 0.5 to 0.25. Quantitative evaluation results for this rollout are reported in Table 9, while the corresponding qualitative diagnostics are shown in Figure 7. The left panel illustrates the close correspondence between target and realised vehicle speed, while the right panel provides a complementary view of the control behaviour used to achieve this tracking quality.

At the same time, this result should not yet be overinterpreted. The current setup isolates speed tracking as the sole optimisation target, and PPO’s performance may change once the additional objectives related to emissions and SOC preservation are activated in the later phases of the methodology.

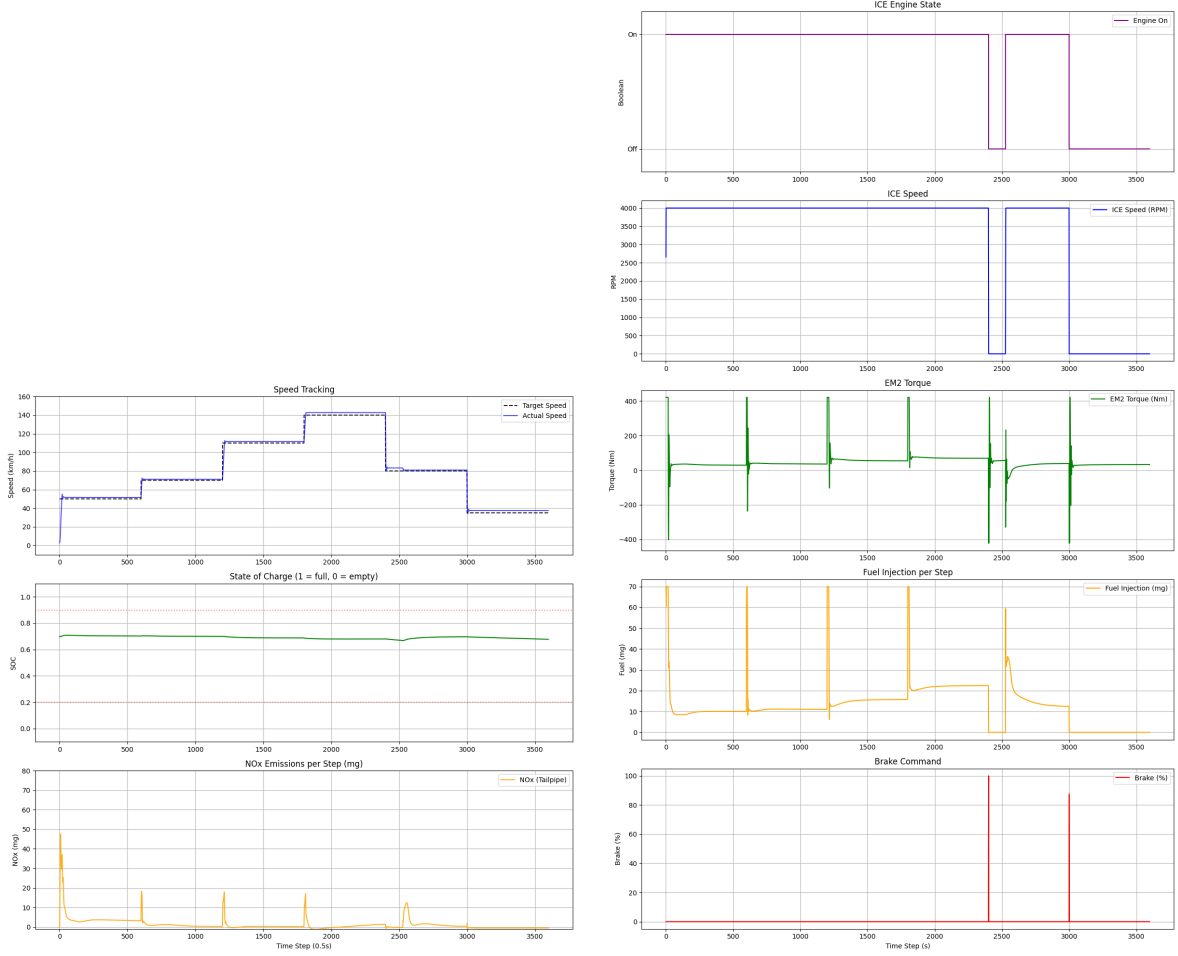


Figure 7: PPO rollout diagnostics. Left: evaluation results for the learned PPO policy. Right: action trajectory and action-distribution view for the same PPO evaluation run.

Metric	Value
Total reward	3498.15
Fuel [g]	45.61
NOx [g]	4.76
MAE speed [km/h]	2.05
RMSE speed [km/h]	3.52
Initial SOC	0.7004
Final SOC	0.6775
Delta SOC	-0.0228

Table 9: Quantitative evaluation results for the PPO rollout shown in Figure 7.

5.2.2 SAC

SAC currently provides the single best run observed so far in terms of mean episodic training reward, reaching 3421.52. Relative to PPO, SAC appears to learn faster in the early part of training, as indicated by the steeper rise in Figure 6. However, this stronger initial progress is accompanied by more pronounced instability: the reward trajectory exhibits a visible dip around 800,000 to 1,000,000 steps before recovering and continuing towards its best value.

This behaviour suggests that SAC is capable in the present control setting, but also somewhat less stable than PPO under the current configuration. The quantitative evaluation results in Table 10 and

the qualitative diagnostics in Figure 8 support this interpretation. The left panel shows that SAC can track the reference trajectory closely, while the right panel reveals flickering behaviour that is not desirable. Between approximately steps 1250 and 1750, the agent rapidly oscillates the ICE speed, EM2 torque, and fuel injection mass. Furthermore, of the four phases in which the ICE is *on*, three last only a few steps. Neither of these observations resembles realistic driving behaviour, although such behaviour may diminish once the agent is trained to optimise the SOC- and emissions-related reward terms introduced in Phase 2. For the present phase, however, the main positive result is that the agent succeeds in speed tracking.

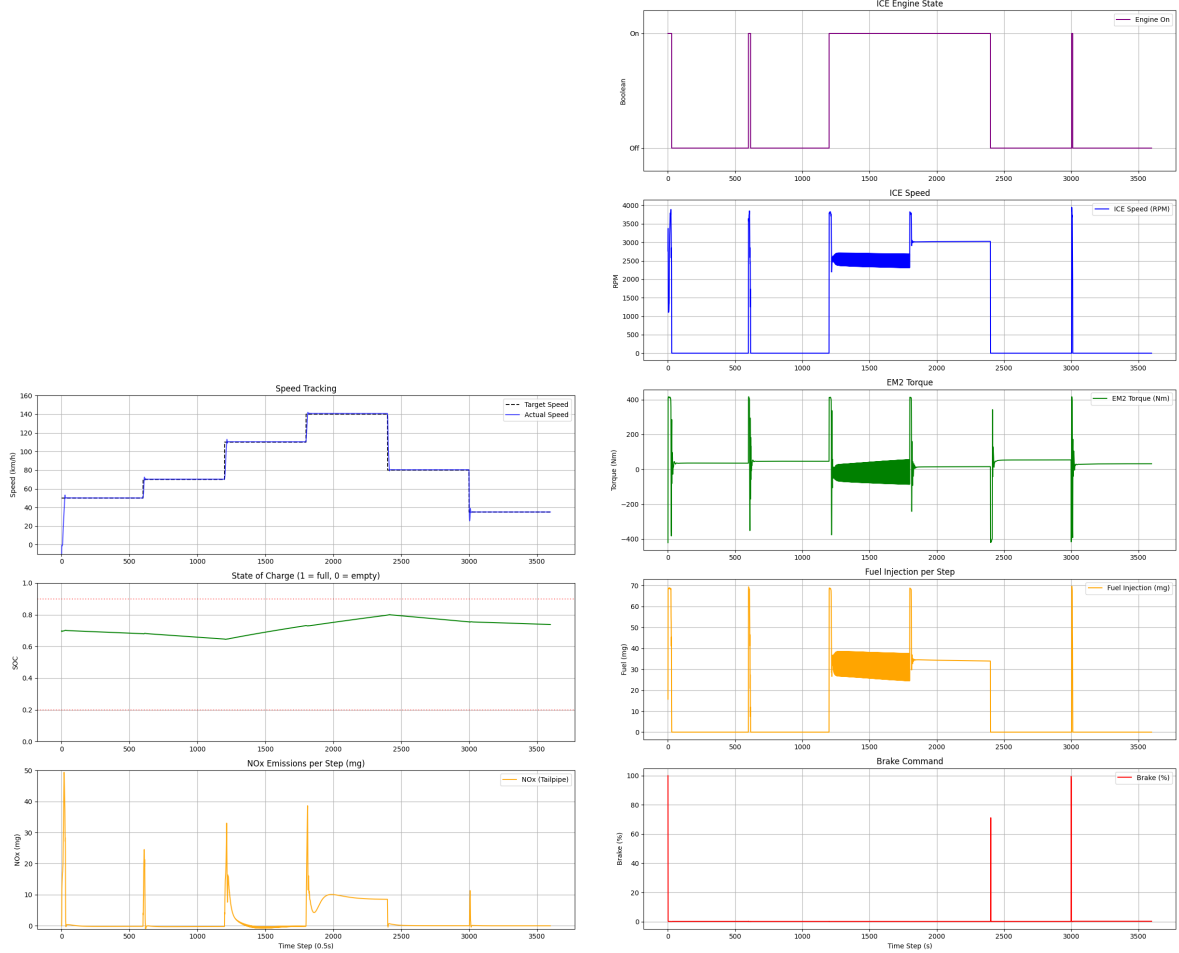


Figure 8: SAC rollout diagnostics. Left: evaluation results for the learned SAC policy. Right: action trajectory and action-distribution view for the same SAC evaluation run.

Metric	Value
Total reward	3513.66
Fuel [g]	43.71
NOx [g]	7.08
MAE speed [km/h]	0.81
RMSE speed [km/h]	4.15
Initial SOC	0.6966
Final SOC	0.7378
Delta SOC	0.0412

Table 10: Quantitative evaluation results for the SAC rollout shown in Figure 8.

5.2.3 TD3

By contrast, TD3 has not yet shown comparably promising behaviour in the current setup. Relative to PPO and SAC, its reward development remains clearly weaker, as can already be seen in Figure 6. The quantitative evaluation results in Table 11 and the corresponding rollout diagnostics in Figure 9 should therefore be read less as evidence of successful control and more as a diagnostic indicator of where learning may currently be failing.

One possible explanation is that the deterministic nature of TD3 interacts unfavourably with the structure of the control problem, especially in the presence of discontinuities induced by engine on/off constraints. Such discontinuities create abrupt changes in the optimisation landscape and thereby make stable learning more difficult. However, this remains a hypothesis rather than a valid conclusion and will require broader experiments and analysis before it can be evaluated more rigorously.

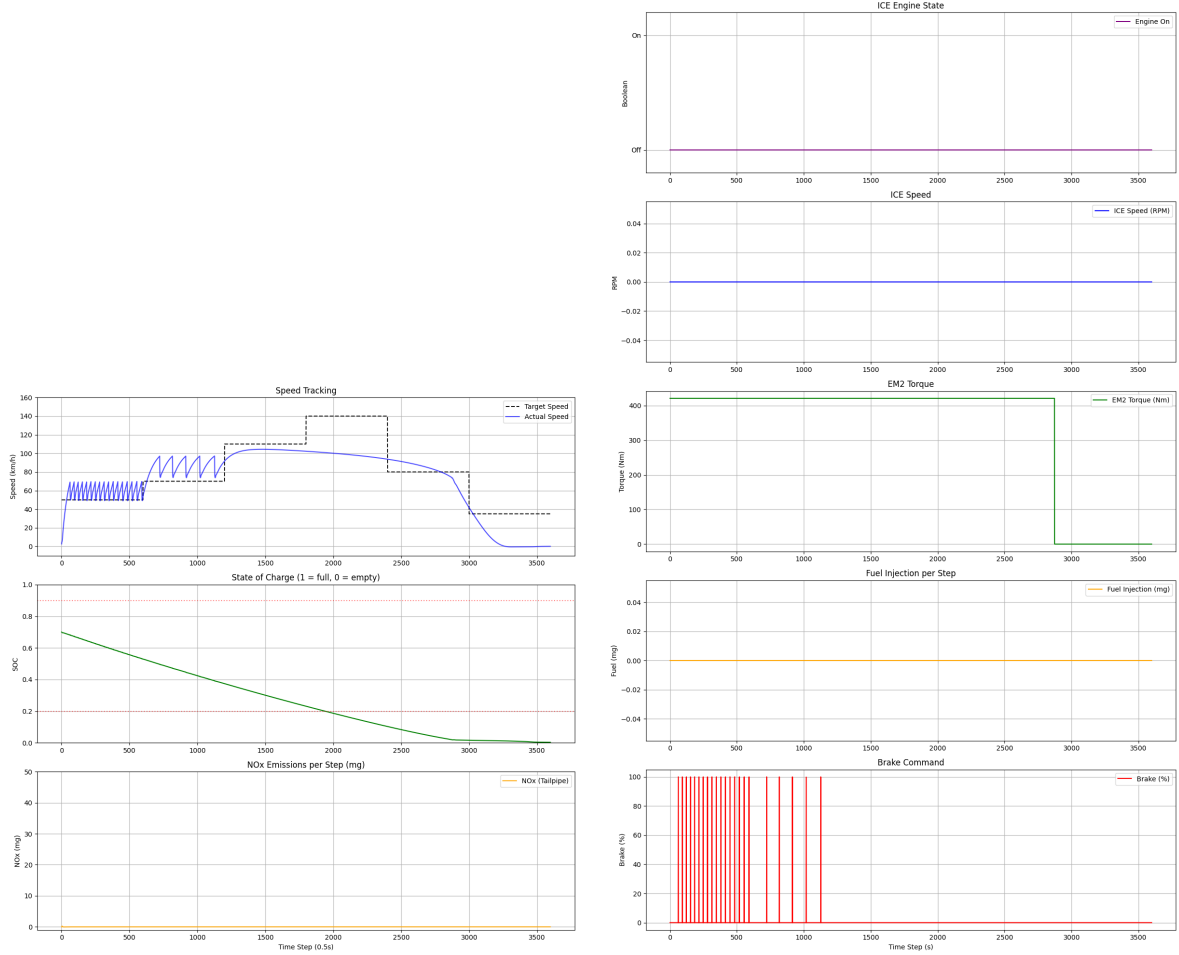


Figure 9: TD3 rollout diagnostics. Left: evaluation results for the learned TD3 policy. Right: action trajectory and action-distribution view for the same TD3 evaluation run.

Metric	Value
Total reward	751.93
Fuel [g]	0.00
NOx [g]	-0.0085
MAE speed [km/h]	19.02
RMSE speed [km/h]	23.68
Initial SOC	0.7004
Final SOC	0.0033
Delta SOC	-0.6971

Table 11: Quantitative evaluation results for the TD3 rollout shown in Figure 9.

6 Conclusion and Next Steps

The current state of the project is best understood as a successful methodological reset. The earlier report structure no longer reflected the actual implementation status, whereas the present phase is organised around updated plant models, a new RL environment, and a broader experimentation workflow. The main bottleneck related to inference time has been resolved, while reward engineering, action semantics, and the staged treatment of multiple objectives have emerged as the dominant challenges at this stage.

The main achievement so far is therefore not a finished final controller, but the establishment of a functioning research platform for developing one. This platform already includes updated recurrent plant models, improved deployment options, a Gymnasium-compatible environment, safety-oriented action logic, staged reward engineering, and a multi-algorithm training pipeline. The project should therefore not yet be judged solely by final control performance, since important questions remain open, including reward tuning, hyperparameter sensitivity, and transfer from simplified tasks to more realistic driving cycles.

Several observations already deserve emphasis. One encouraging preliminary finding is that both PPO and SAC appear to maintain the SOC within a comparatively narrow range, even though explicit SOC regulation is not yet the central optimisation target in Phase 1. This still has to be validated through statistically meaningful multi-seed experiments, but it is potentially important because future NO_x reduction will likely depend on whether emissions can be improved without causing unrealistic battery depletion.

A further point concerns architectural comparability. In the present study, the default implementations from **Stable-Baselines3** are retained in order to keep the setup simple, transparent, and reproducible. However, this also means that the underlying policy and value-network architectures are not fully matched across algorithms. In particular, PPO uses the default two-layer architecture with 64 units per layer, whereas SAC and TD3 use two layers with 256 units per layer. The representational complexity available to the different algorithms is therefore not identical, which must be taken into account when comparing their performance. Nevertheless, at the current stage of development, adhering to the default implementations remains a practical and methodologically defensible choice.

The next stage of the project is to build on this infrastructure more systematically: first by improving the robustness of speed control, then by reintroducing emissions and SOC optimisation in a controlled manner, and finally by identifying which algorithmic configuration offers the best trade-off between performance, stability, and computational efficiency. In parallel, two implementation issues identified during the preparation of this report will need to be addressed directly in the next iteration. First, the observations are not yet scaled correctly in the current environment implementation. Second, the current environment includes substantially more observations than in the earlier work assessment, which changes both the information structure of the problem and the learning difficulty. Both issues should be resolved before drawing stronger conclusions from cross-algorithm comparisons.

Another priority for the next iteration is a more detailed analysis of the generated diagnostics, especially the state-visitation and state-action coverage plots. These outputs are expected to provide a deeper understanding of how the learned policies use the admissible state space and whether apparently strong scalar performance is supported by physically plausible control behaviour (e.g. mapping action on the engine map).

At present, progress is also constrained by the lack of access to the compute cluster. This affects both, statistically meaningful multi-seed experiments and the planned hyperparameter optimisation with Optuna. Once cluster access is available, these experiments should make it possible to move from promising preliminary findings to more robust and defensible conclusions.

Overall, the present phase has established the technical and methodological foundations for the remainder of the project. The next report iteration should therefore be able to shift the focus from infrastructure building and problem reformulation towards statistically grounded evaluation, deeper behavioural analysis, and a clearer assessment of which RL configuration is most suitable for the control task.